MERCHANT GROWTH COPILOT · HACKATHON PROTOTYPE

DukaanPulse *AI*



Turn payment signals into **profitable merchant actions**.

multi-agent | local LLM · rules fallback | evidence-checked claims | approval-gated | synthetic data

Evidence-backed decisions for small merchants — with the merchant in control

Paytm Build for India AI Hackathon · Merchant Growth AI track

Live demo dukaanpulse-ai.vercel.app **Code** github.com/Thebinary110/dukaanpulse-ai

Working prototype on synthetic data for a fictional bakery. Not affiliated with or endorsed by Paytm.

PROBLEM STATEMENT

Sales are slipping. Should the bakery discount everything?

−13.0%

revenue, last 30 days vs the 30 before

₹7.91L vs ₹9.09L

−21.6%

morning revenue — the biggest fall

20 Sep – 19 Oct

34 of 209

regulars away far longer than their own habit

median 30 days vs 2.4 usual

OBSERVED All figures come from the app's synthetic payment data for a fictional bakery.



The owner has the data. What they lack is a trustworthy answer to: "What is worth doing, for whom, and what will it cost?"

PROBLEM STATEMENT

Dashboards show change. Chatbots sound sure. Neither decides.

Analytics dashboard

- Shows revenue fell
- Leaves the “so what?” to the owner
- No cost or trade-off of acting

Generic AI chat

- Fluent, confident answers
- Can invent numbers, segments or causes
- Can be pushed to “just send it”

DukaanPulse AI

- Finds who changed and why it matters
- Prices each response, incl. doing nothing
- Every claim checked; merchant approves

For a decision that costs real money, “sounds right” is not good enough — every number and claim must be traceable.

PROPOSED SOLUTION

One loop: from a payment signal to a measured decision

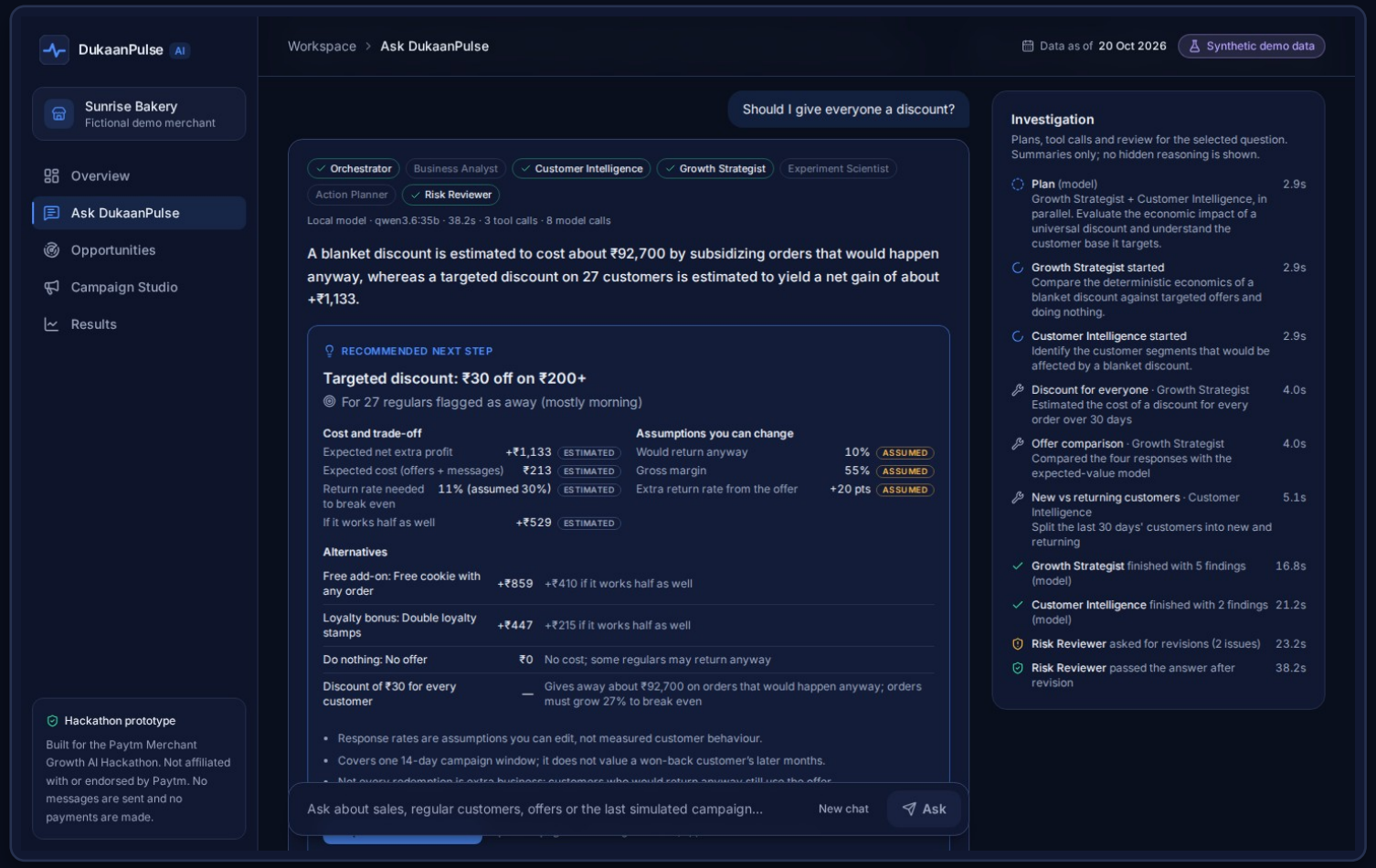


Results feed the next decision: repeat with a larger test, revise the offer, or stop

Agents reason. Tested calculations produce every number. **The merchant decides.**

PROPOSED SOLUTION

Live: “Should I give everyone a discount?”



REAL SCREEN

Local Qwen model via Ollama · ~25 s average per question · instant rule-based mode if needed

Agents work in parallel

Growth Strategist and Customer Intelligence investigate together; the timeline shows each tool call.

A decision, not a chart

Recommended action, who it targets, cost, assumptions and alternatives.

Checked before shown

An independent reviewer removed unsupported claims from this answer.

PROPOSED SOLUTION

The numbers behind it: target the regulars, don't subsidise all

₹30 off for every customer

₹92,700

given away a month on orders that would happen anyway — if every one of the last 30 days' 3,090 orders used it

+27% orders needed just to break even (≈837 extra orders)

ESTIMATE · ASSUMES 55% MARGIN

Recommended: ₹30 off ₹200+, 27 regulars

+₹1,133 expected net extra profit

₹213 expected cost (offers + messages)

11% vs 30% return rate to break even vs assumed

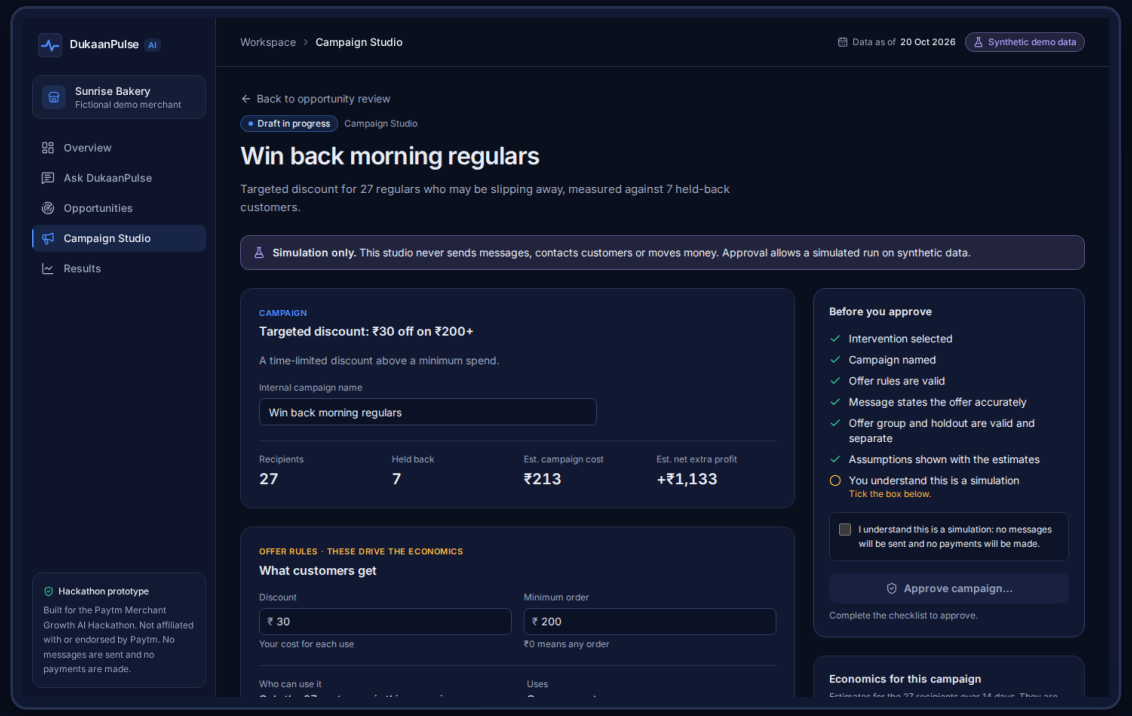
+₹529 still positive if it works half as well

ESTIMATE · ASSUMES +20 PT UPLIFT

Read this honestly. Different scopes: the blanket figure covers all orders over 30 days; the targeted offer covers 27 flagged regulars (7 more held back) over a 14-day campaign. Both are model estimates from editable assumptions — 10% would return anyway, 85% of returners redeem, 1 later full-price visit — not measured results. Alternatives shown in-app: free add-on +₹859, loyalty bonus +₹447, do nothing ₹0.

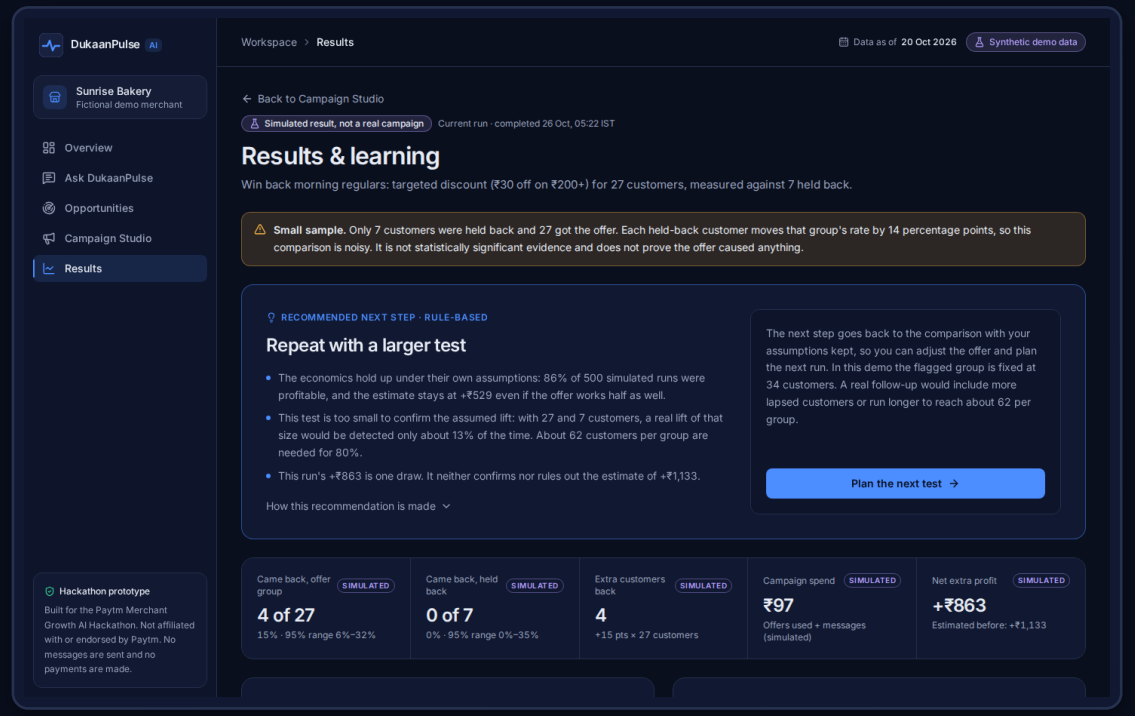
PROPOSED SOLUTION

Act, then measure honestly — even when evidence is thin



Approve the exact offer; no edits after approval

Message must state the offer's real terms; a checklist gates approval.



Simulated result vs 7 held-back customers

27 vs 7 gives only ~13% power to detect the assumed lift; ~62 per group needed. The app says "repeat with a larger test" instead of claiming a win.

SIMULATED

How it works: agents reason, tools calculate, a reviewer checks



Tech stack TypeScript · React 19 · Vite · Tailwind CSS · Recharts | Qwen 3.6 35B on Ollama via a 2-route server proxy | Vitest · Playwright · axe-core

Our USP: the whole loop, evidence-checked and merchant-controlled

Prices every option	Break-even and a half-as-good downside for each offer — and doing nothing — before anyone acts.
Measures against a holdout	Results are read against held-back customers, and the app says when the sample is too small to tell.
Checks what AI claims mean	Right customers, right window, right direction, estimate vs fact vs simulated, no causes stated as proof.
Approval binds the offer	Agents can only propose. The merchant approves the exact terms; nothing changes after approval.
Local-first and resilient	Runs on a local open model on our own hardware, with an instant, labelled rule-based fallback.

Why the combination matters

Each piece exists somewhere. Together they let a merchant act on AI advice without trusting it blindly.

What should I do?

→ options priced, incl. doing nothing

What will it cost?

→ break-even and downside

Did it work?

→ read against a holdout

All five are implemented and tested in the prototype (see appendix).

USP

Trust by design: no invented numbers, segments or actions

Numbers only from tools

Agents write {{references}}; the app fills in figures from tested calculations. Typed digits are rejected.

Claims checked for meaning

Right customers, right time window, right direction, estimate vs fact vs simulated, no causes stated as proof.

Merchant in control

Agents can only propose. Approval binds the exact offer; nothing is ever sent or paid in this prototype.

A real failure the reviewer now catches (regression-tested)

✗ Model wrote: “Among 209 regular **morning** customers, 34 showed an unusually long absence.”

The 209 is correct — but it counts all regulars, not morning ones. Rejected: segment mismatch (S1).

✓ Shown instead: “34 of 209 regular customers have been away much longer than usual.”

USP

How we know it works — and what that doesn't prove

414

unit tests

domain, state machine, agents, reviewer

73

browser tests

full workflow, 3 consecutive runs

12 / 12

live-model cases

local Qwen; every claim shown grounded

12 / 12

bad claims rejected

planted claims, model and rules paths

Adversarial cases tested

- Wrong customer segment
- Wrong time window
- Correlation as proof
- Simulated called real
- Invented or missing references
- Contradicting a tool
- Missing cost assumptions
- Unanswerable question
- Conflicting recommendation
- Attempted approve / send

What this proves: answers are grounded, scoped and honestly labelled on tested synthetic cases, and agents cannot act. **What it does not:** that any recommendation increases real merchant revenue. That needs a real, adequately sized randomized test.

Impact & benefits: better decisions now, revenue proof next

For the merchant

SHOWN IN PROTOTYPE

Avoids blind storewide discounts: ₹30 off for all needs +27% orders to break even

SHOWN IN PROTOTYPE

One clear decision in plain words: who, what it costs, the risk and the alternatives

SHOWN IN PROTOTYPE

Knows whether it worked — or that the test was too small to tell

For their customers

SHOWN IN PROTOTYPE

Fewer, more relevant offers: 27 lapsed regulars contacted, not every customer

SHOWN IN PROTOTYPE

Messages must state the offer's real terms

For the platform

SHOWN IN PROTOTYPE

Merchant-facing AI that does not invent figures (on tested cases)

EXPECTED — TO TEST

A weekly reason for merchants to come back to the app

EXPECTED — TO TEST

Holdout results across merchants show which offers really work

Not yet shown: revenue impact for real merchants. "Shown" means demonstrated on synthetic data; proving real impact is the pilot's job.

Business model (proposed): a decision layer in the payment app

PROPOSED — NOT VALIDATED

First merchants

Repeat-visit businesses already taking digital payments — bakeries, cafés, kiranas — where regulars drive revenue (91% of revenue from returning customers in our synthetic shop).

Distribution

Surfaced where merchants already check payments: a weekly "one decision worth making" with evidence, not another dashboard.

How it could earn — options to test

Free weekly insight to drive engagement; a paid campaign-and-measurement tier; or a fee linked to measured incremental profit. No pricing validated yet.

Cost to serve

Deterministic tools do the maths; a compact local model or a hosted model writes the narrative — and a rules-only mode costs nothing to run.

Assumption the pilot must test: merchants act on a weekly decision, and the campaigns they approve pay back when measured against a holdout.

Next: prove it with real merchants, measured properly

1

Pilot data

0–1 month

Consented payment data from a small set of repeat-visit merchants; detector and tools re-run unchanged.

2

Real holdout tests

1–3 months

Randomized offer vs holdout, sized for power (≈62+ per group per test); merchant approves every campaign.

3

Measure what matters

3–4 months

Incremental return rate and margin per merchant — reported with uncertainty, including no-effect results.

4

Scale what works

after evidence

Keep the offers that pay back; add opportunity types beyond lapsed regulars.

Our ask

- A pilot with a handful of merchants
- Guidance on consented data access
- Mentors to shape the experiment design

Timelines are our plan, not commitments from anyone else.

Evidence, not guesses.

Decisions, not dashboards.

And the merchant always has the final say.

DukaanPulse AI
Working prototype · synthetic data · local model + rules fallback · not affiliated with Paytm

Live demo
dukaanpulse-ai.vercel.app
Code
github.com/Thebinary110/dukaanpulse-ai

Appendix: where the numbers come from

Claim	Type	Source / method
Revenue −13.0%, morning −21.6%, 34 of 209 regulars	**Observed**	Seeded synthetic data (180 days, seed 20261020); comparePeriods, revenueByDaypart, lapse detector
₹92,700 · +27% (≈837 orders)	**Estimate**	3,090 orders × ₹30 if all redeem; break-even at ₹256 × 55% − ₹30 per extra order
+₹1,133 · ₹213 · 11% · +₹529	**Estimate**	campaignEconomics for 27 customers, 14 days; baseline 10%, uplift +20 pts, 85% redemption, 55% margin
4 of 27 vs 0 of 7 · +₹863	**Simulated**	Rehearsed in-app run (seeded by approved configuration); not real customers
~13% power · 62 per group	**Estimate**	Two-proportion normal approximation, 10% → 30%, α = 0.05, 80% power
414 · 73 · 12/12 · 12/12	**Test result**	Re-run on the current commit; live cases are synthetic scenarios

Full claim-by-claim mapping: Pitch_Sources_and_Claims.md